1 Scatter plot
2 Boxplot
3 Histogramas
4 Densidad
5 Diagrama de barras
6 mejorar ggplot
7 Ejercicios propuestos

Clase prácticas. Día3: Gráficas en R

Juan A. Ferrer-Bonsoms & Gonzalo Fernández-Duval

Estas prácticas en R engloban dos enfoques:

- Teórico: en el que se detalla como se modeliza matemáticamente un fenómeno aleatorio.
- Práctico: "visualizar" algunos fenómenos aleatorios a partir de un número grande de observaciones. Esto nos permitirá "visualizar" la teoría.

Lo característico de los fenómenos aleatorios es que sus resultados se repiten (existe regularidad en los resultados), lo que permite modelizarlos matemáticamente. Cuando se tienen multitud de resultados obtenidos de repeticiones del fenómeno en cuestión, esa "regularidad" se puede apreciar.

Pero para hacerlo e identificar los patrones de un conjunto de datos, es necesario agrupar las observaciones y representar los datos agrupados.

En concreto, en esta práctica vamos a ver las siguientes gráficas y cuando aplicarlas.

- scatter plot.
- boxplot.
- density plot.
- histogram plot.
- bar plot.

Para cada gráfica vamos a utilizar dos tipos de gráficas:

- gráficas de R base, que son las gráficas más básicas de R.
- gráficas utilizando la librería `ggplot2`, que permite realizar gráficas más elegante, pero cuya programación puede ser menos intuitiva.

En el examen podrán utilizar R base o `ggplot2`.

1 Scatter plot

El scatter plot es la gráfica más sencilla que vamos a ver. Este plot muestra los datos en su conjunto, representándolos individualmente.

Este tipo de gráficas se utilizan sobretodo para ver la relación entre **dos variables continuas**.

Vamos a ver un ejemplo utilizando la base de datos de `cars` que viene incorporada en `R`.

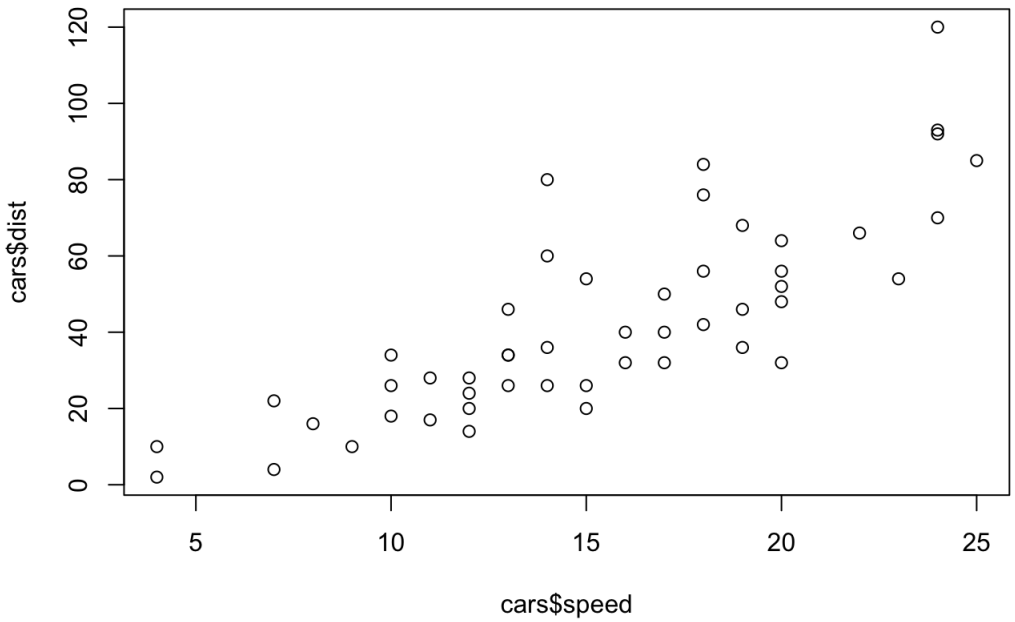
```
data(cars)
# dim(cars) #50 filas y 20 columnas
```

Esta base de datos tiene la distancia que recorre un coche en la frenada en función de la velocidad.

Además, como ya se ha mencionado antes, se puede utilizar este tipo de gráficas para **relacionar variables continuas**. En este caso, queremos dibujar una gráfica que relacione la distancia recorrida en el frenado con la velocidad:

R base

```r
plot(x=cars$speed,
     y=cars$dist)
```



Como se puede observar, en la función `plot` tenemos dos variables de entrada:

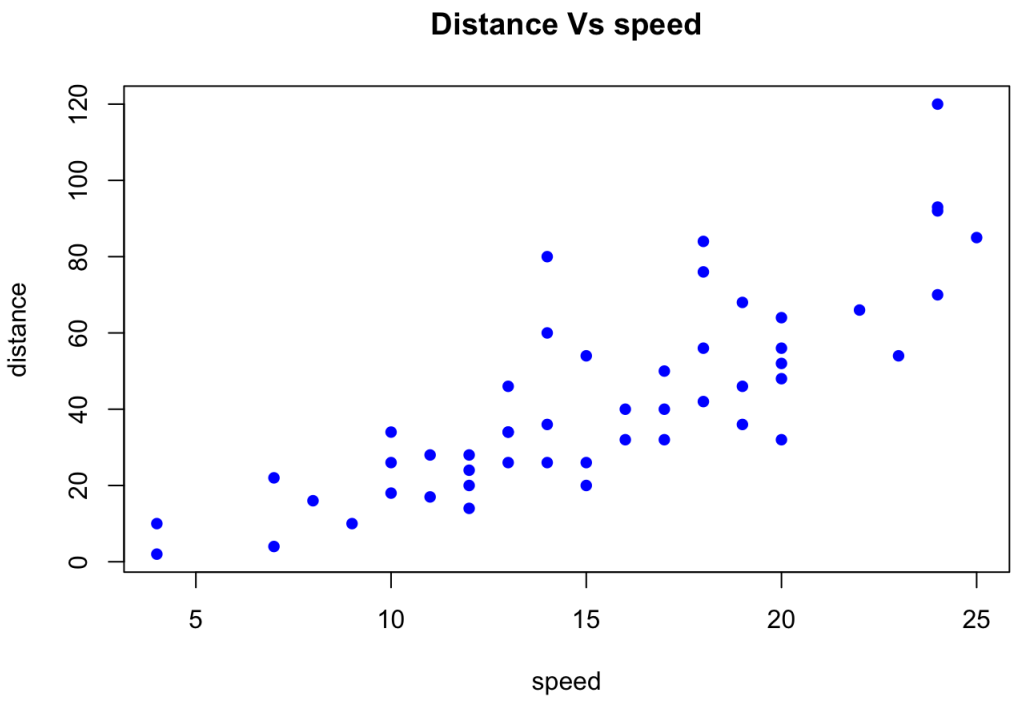
- `x`: un vector que indica la coordenada x.
- `y`: un vector que indica la coordenada y.

Además de estos parámetros, existen otros parámetros que sirven para modificar características de la gráfica:

- **main**: Título del plot.
- **xlab**: Título para el eje x.
- **ylab**: Título para el eje y.
- **xlim**: Rango de valores para el eje x.
- **ylim**: Rango de valores para el eje y.
- **col**: Color o colores a utilizar.
- **pch**: Tipo de puntos.
- **type**: tipo de gráfica. “p” dibuja puntos (valor por defecto). “l” dibuja lineas, “b” dibuja tanto lineas como puntos. Existen más tipos.
- **lty**: Tipo de línea.
- etc...

Sabiendo esto, podemos mejorar la gráfica anterior:

```r
plot(x=cars$speed,
     y=cars$dist,
     col="blue",
     pch=16,
     main = "Distance Vs speed",
     xlab = "speed",
     ylab = "distance")
```



ggplot2

con la librería `ggplot2` lo primero que tenemos que hacer es cargar la librería:

```
library(ggplot2)
```

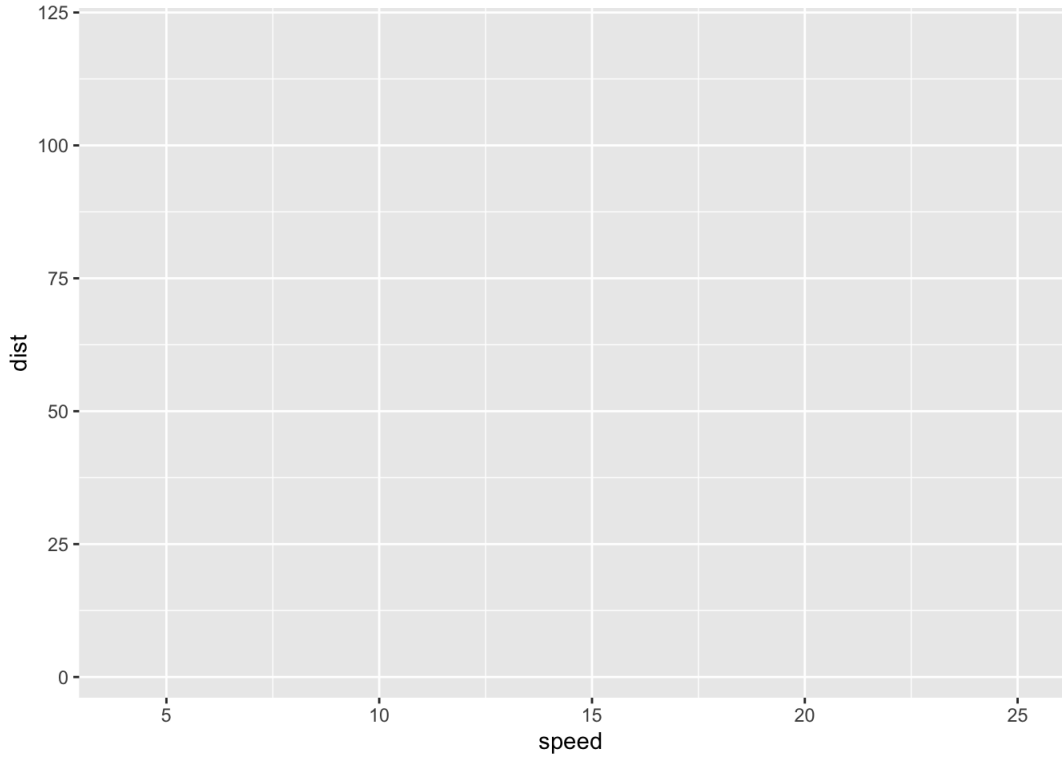
Para crear la gráfica tenemos que ejecutar la función `ggplot` que tiene los siguientes parámetros:

- data= el data.frame donde tengo los datos
- aes(...)= para indicar que columna del data.frame sea el eje x, el eje y, etc.

En nuestro caso, data será igual a `cars` y queremos en el eje x la velocidad y en el eje y la distancia, por lo tanto pondremos `aes(x=spped,y=dist)`

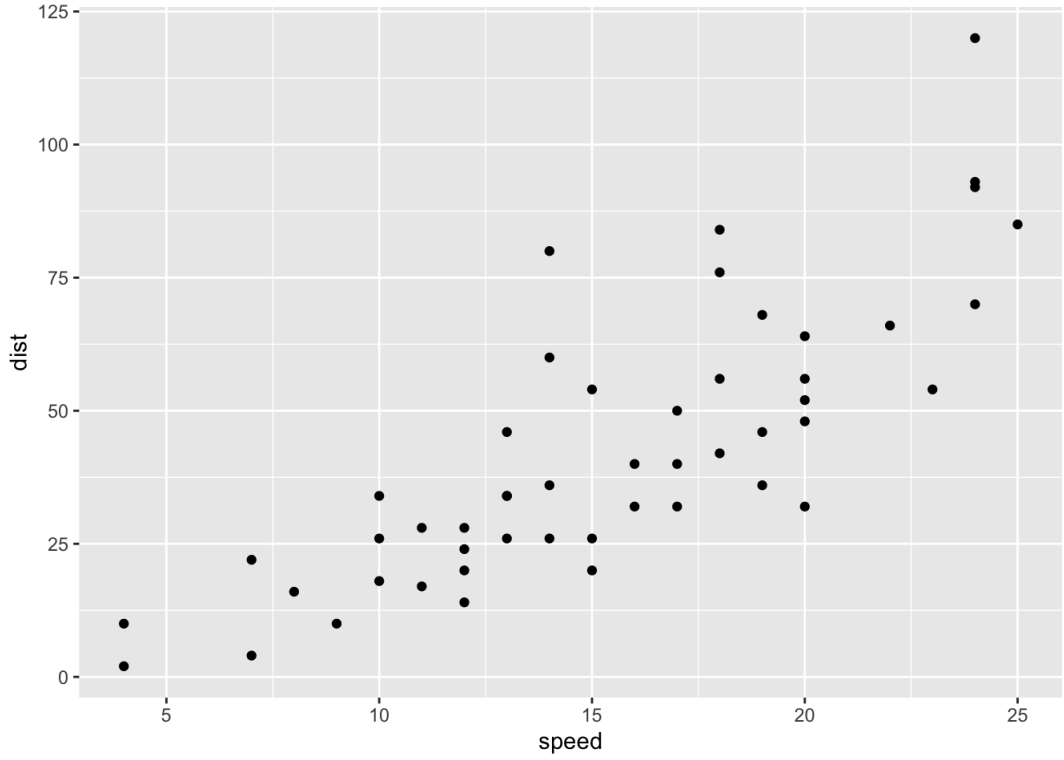
Si ejecutamos el siguiente código se creará una gráfica vacía, sin nada dibujado.

```
ggplot(data=cars,aes(x=speed,y=dist ))
```



Esto es así porque todavía no le hemos indicado a ggplot el tipo de gráfica que queremos. En nuestro caso, como queremos un scatter plot, la función que tenemos que llamar es `geom_point()`. Para ejecutar `geom_point()` tenemos que añadirlo al código anterior con un `+`

```
ggplot(data=cars,aes(x=speed,y=dist ))+
  geom_point()
```



Ahora si, ya tenemos el scatter plot realizado con la libraría `ggplot2`.

En la sección de la práctica "mejorar ggplot" podéis encontrar una explicación más extensa y detallada sobre `ggplot2`.

2 Boxplot

Un boxplot muestra la dispersión de un conjunto de datos teniendo en cuenta sus valores de posición (cuartíles, mediana y rango).

Su principal ventaja es que de un vistazo revela la mediana y el rango intercuartil (el rango en el que se encuetra el 50% de los datos más centrados). Además, permite detectar anomalías o errores en los datos.

Vamos a ver un ejemplo con la base de datos `CO2`.

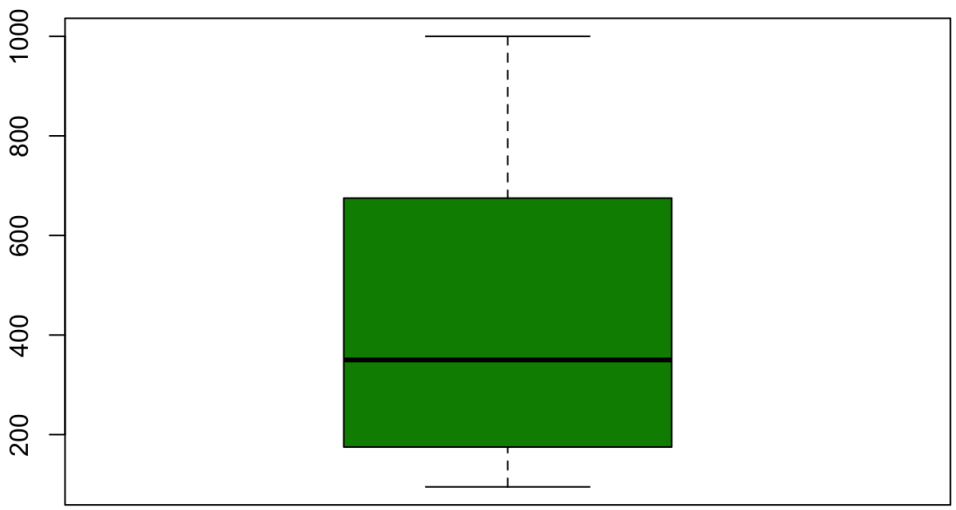
```
data("CO2")
```

Esta base de datos muestra información sobre el consumo y emisiones de CO2.

Con el boxplot podemos ver como se destribuyen los valores de la columna `conc`:

R base

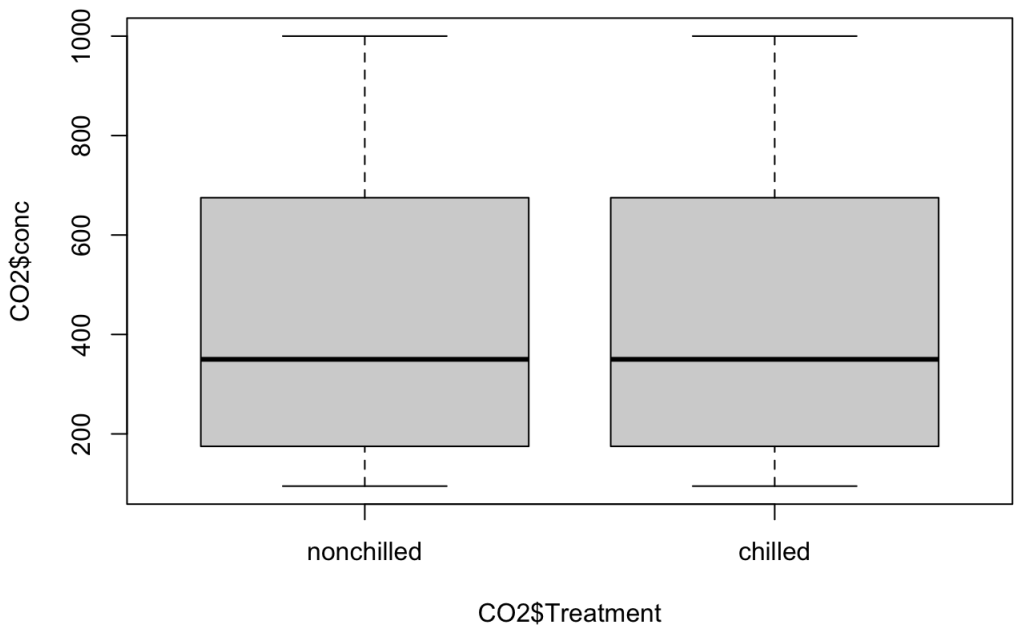
```
boxplot(CO2$conc,col="green4")
```



Por otro lado, puede que nos interese representar para cada tipo de `treatment` como se distribuye los valores de la columna `conc`. Para ello se puede utilizar el operador `~` de la siguiente manera: `boxplot(CO2$conc~CO2$Treatment)`.

Con este código lo que le estamos pidiendo a R es: "Dibuja para cada posible valor que hay en Treatment un boxplot con los valores de conc"

```
boxplot(CO2$conc~CO2$Treatment)
```

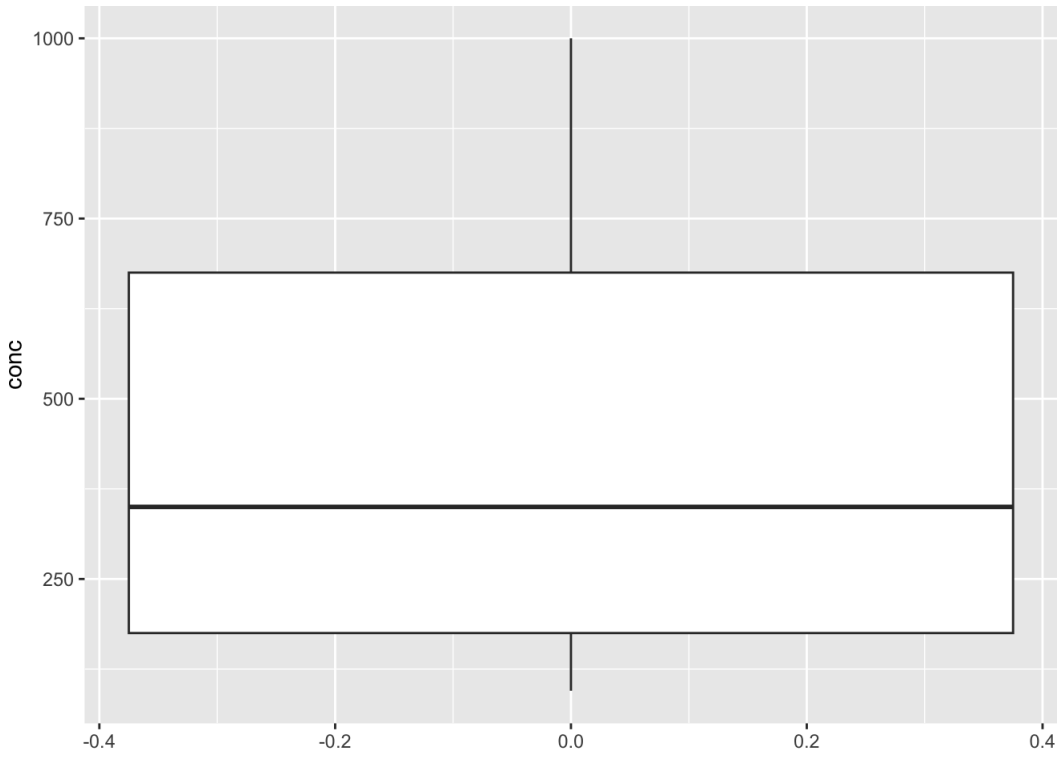


Por lo tanto, lo que teneos en el primer boxplot es el boxplot correspondiente a los valores de `conc` para las filas donde la columna `Treatment` vale "nonchilled". En el segundo boxplot corresponde a los valores de `conc` donde tenemos en la columna `Treatment` el valor "chilled".

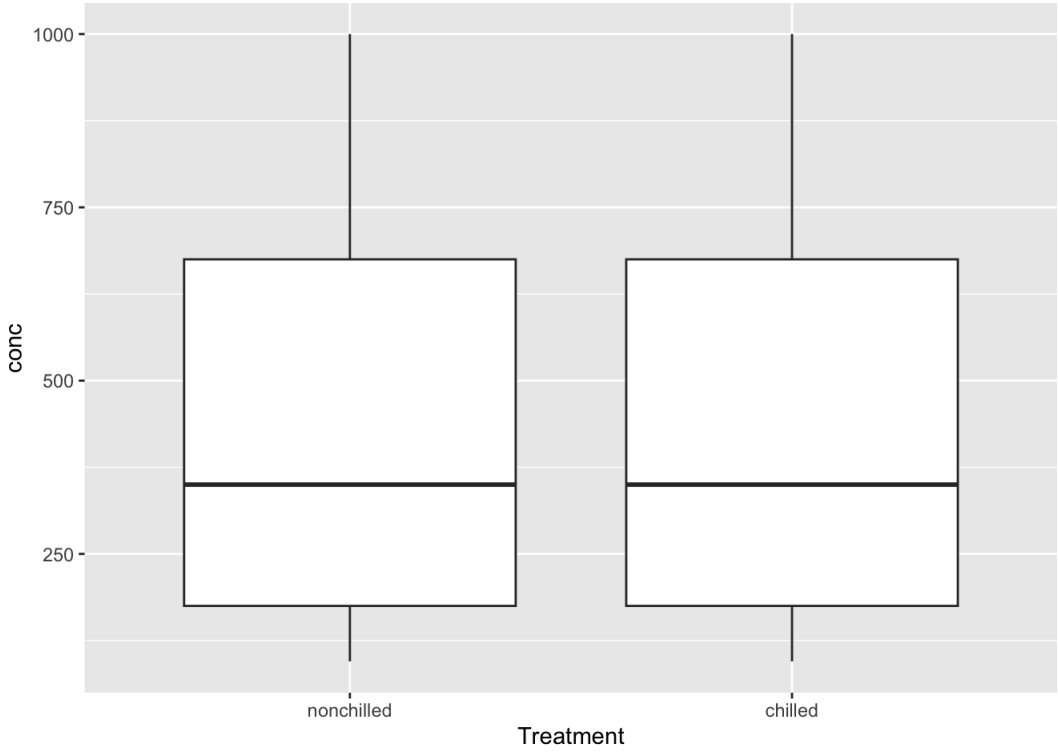
Por lo tanto, el boxplot se utiliza para **relacionar variables continuas con variables discretas**. En nuestro caso, la variable continua es conc y la variable discreta es Treatment.

ggplot2

```
ggplot(data=CO2,aes(y=conc))+
  geom_boxplot()
```




```
ggplot(data=CO2,aes(y=conc,x=Treatment))+
  geom_boxplot()
```



3 Histogramas

Los histogramas muestra la forma que sigue un conjunto de datos y su dispersión. Es la representación que se utiliza para **variables continuas**.

Con los histogramas podemos ver fácilmente la media, la variabilidad de los datos y la forma de la distribución. Además, permite detectar anomalías (datos que proceden de distribuciones distintas), los valores que aparecen con más frecuencia, etc.

Vamos a ver un ejemplo con los datos de la Behobia del 2021. Para ello tiene que cargar los datos `behobia_2021.RData`. Para cargar estos datos tienen que ejecutar la siguiente línea de código:

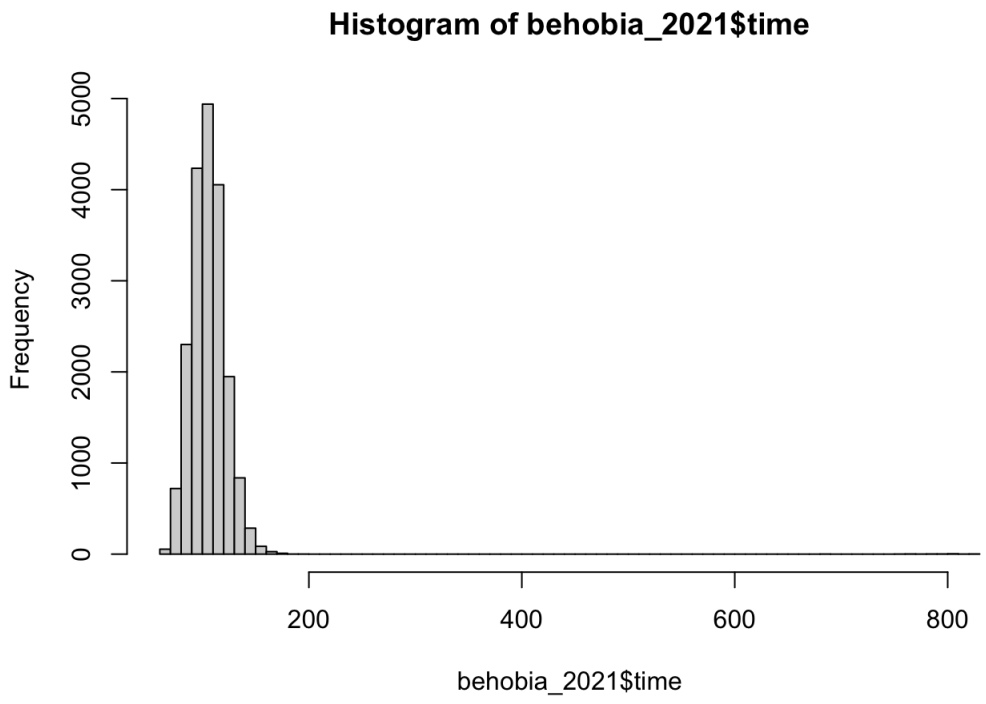
```
load("./behobia_2021.RData")
```

NOTA: los archivos .RData son archivos donde se puede guardar el "environment" o algunas variables que tenemos en nuestra sesión de R. Al cargar un .RData, cargamos en nuestro "environment" esas variables.

R base

Con la función `hist` vamos a visualizar como se distribuye el tiempo total que tardan los corredores en hacer la Behobia:

```
hist(x = behobia_2021$time,breaks = 100)
```



Esta función tiene 2 valores de entrada:

- `x`: el vector con el que quiero dibujar mi histograma. En este caso, el vector es `behobia_2021$time`.
- `breaks`: el número de "barras" que quiero que haga.

En la gráfica se ve claramente que los tiempos están entre 0 y 200 minutos. Entonces, ¿por qué el eje x va desde 0 a 800?

Esto es porque en `behobia_2021$time` hay datos que están por encima de 600. Son tan pocos que no se aprecian en el eje y.

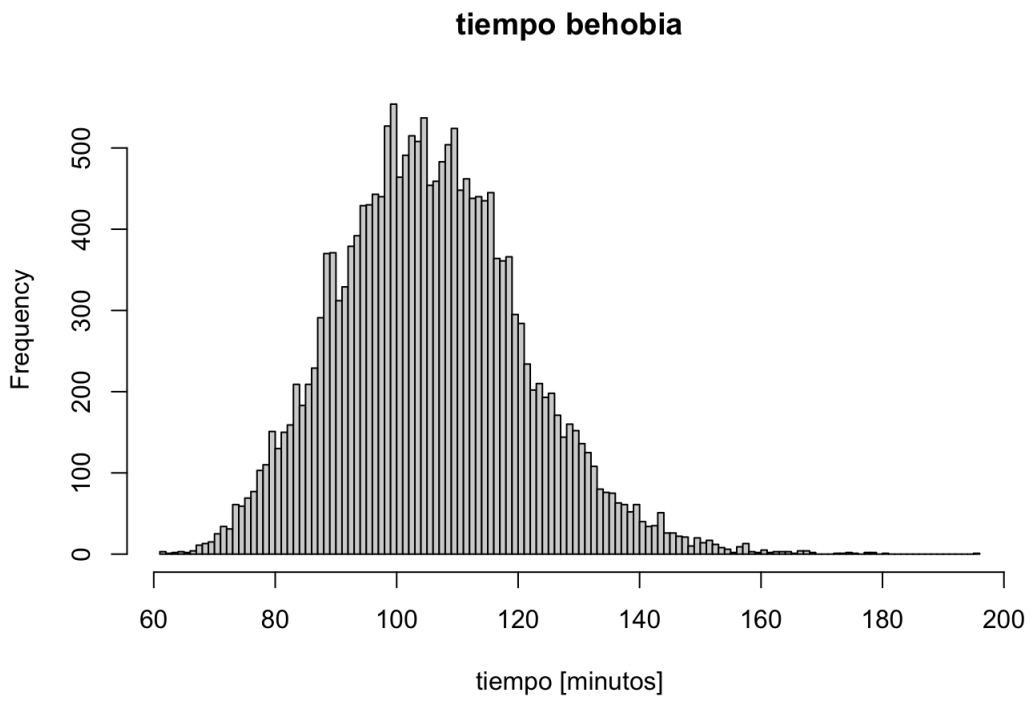
Estos casos son claramente datos que están mal y no debemos considerarlos. Esta es una de las ventajas del histograma (también lo hubieramos visto con un boxplot), que nos permite ver valores que están fuera de lugar.

Con el siguiente código vamos a eliminar los casos que están mal:

```
rrx <- which(behobia_2021$time>600)
#rrx me indica en que filas tengo un valor en "time" mayor a 600
#por lo tanto, queremos todas las filas, menos las correspondientes a rrx.
behobia_2021_2 <- behobia_2021[-rrx,]
```

Con el nuevo data.frame, vamos a dibujar el mismo histograma:

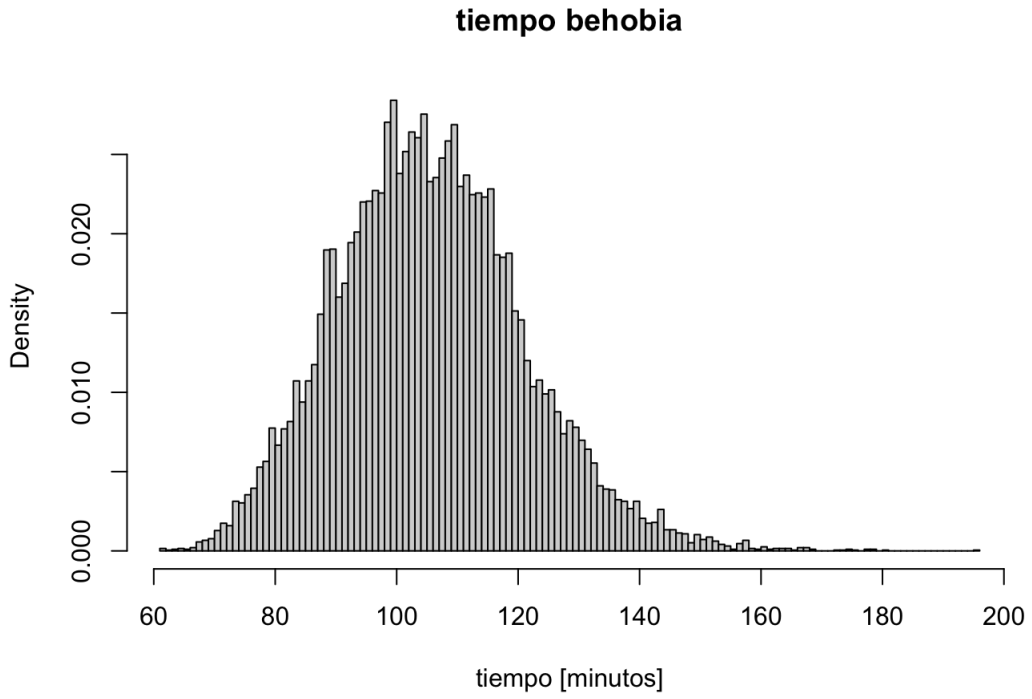
```
hist(x=behobia_2021_2$time,breaks = 100,main="tiempo behobia",xlab="tiempo [minutos]")
```



En esta gráfica, en el eje y tenemos representado el total de casos para cada rango en el eje x. Es decir, entre 100 y 102 tenemos algo más de 500 corredores.

Con el parámetro `freq` podemos cambiar para que en el eje y nos dibuje una **estimación de la función de densidad**. Lo único que cambia es el valor en el eje y, la forma de la gráfica sigue igual.

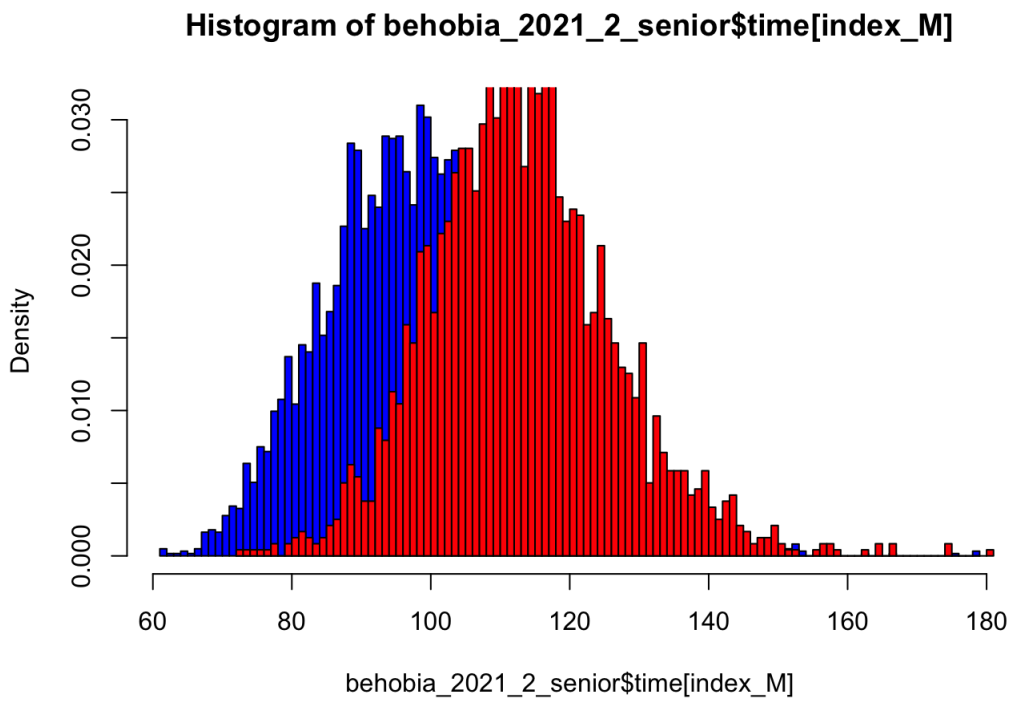
```
hist(x=behobia_2021_2$time,breaks = 100,main="tiempo behobia",xlab="tiempo [minutos]",fre
q=FALSE)
```



Se puede utilizar histograma igual que el boxplot para **relacionar variables continuas con discretas**. Pero esa gráfica no suele quedar muy bien.

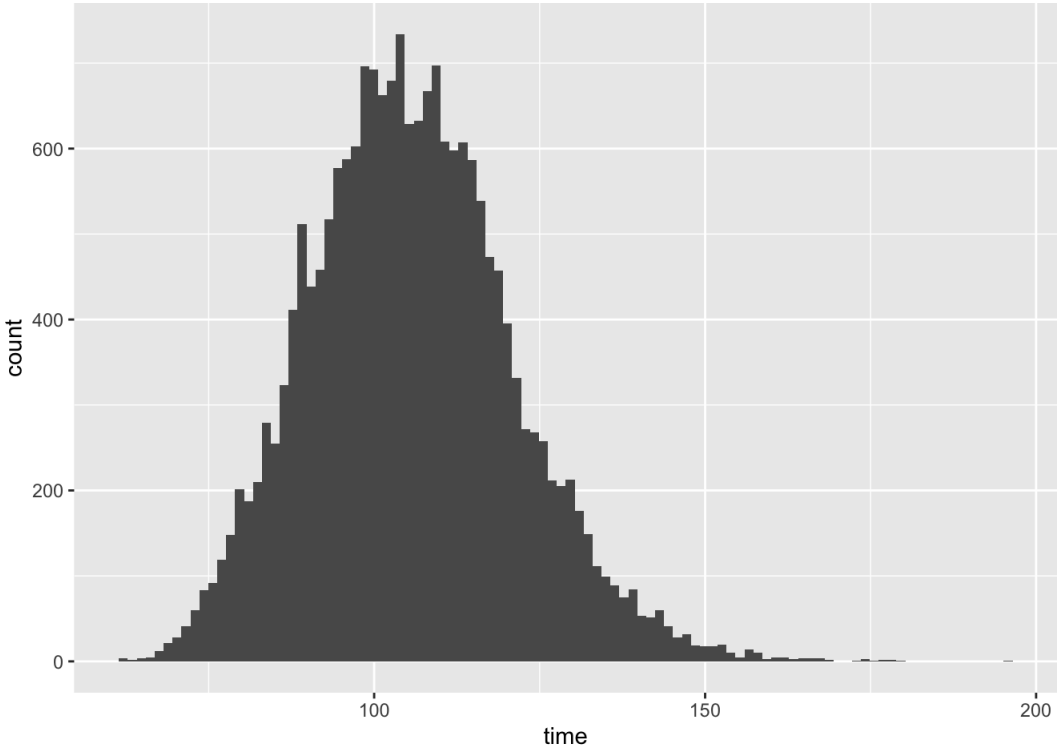
Como ejemplo, vamos a dibujar un histograma con el tiempo de los corredores de la categoría “SENIOR M” y en la misma gráfica los corredores de la categoría “SENIOR F”:

```r
#primero me voy a quedar con los corredores SENIOR:
indice_senior = which(behobia_2021_2$categoria=="SENIOR F" | behobia_2021_2$categoria=="S
ENIOR M")
behobia_2021_2_senior = behobia_2021_2[indice_senior,]

# despues voy a ver en mi nuevo data.frame
# quienes son SENIOR M y quiees son SENIOR F
index_M = behobia_2021_2_senior$categoria=="SENIOR M"
index_F = behobia_2021_2_senior$categoria=="SENIOR F"
hist(behobia_2021_2_senior$time[index_M],breaks = 100,freq = F,col="blue")
hist(behobia_2021_2_senior$time[index_F],breaks = 100,add=TRUE,freq=F,col="red")
```



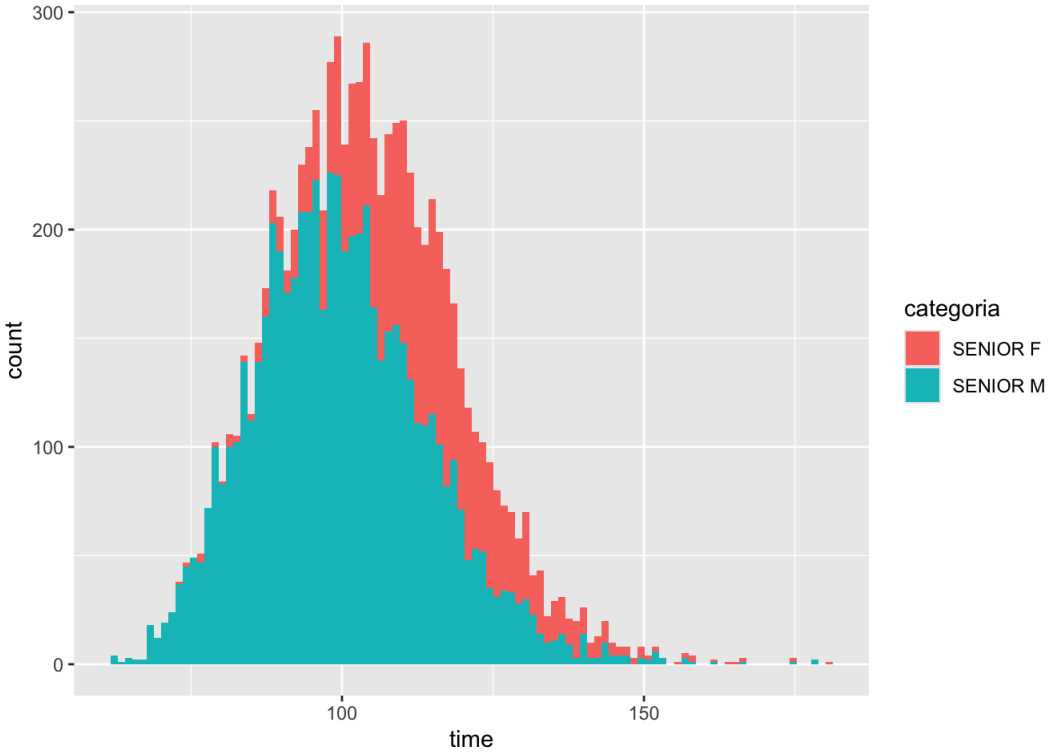
ggplot2

```r
ggplot(data=behobia_2021_2,aes(x=time))+
  geom_histogram(bins = 100)
```



Para dibujar 2 histogramas, en el parámetro aes le podemos indicar que el color de las barras dependa del valor de la columna `categoria`. Esto se hace con `fill = categoria`. En algunas gráficas de ggplot el color se indica con `col` y en otras con `fill`. Al final de la práctica veremos esto.

```
ggplot(data=behobia_2021_2_senior,aes(x=time,fill=categoria))+
  geom_histogram(bins = 100)
```



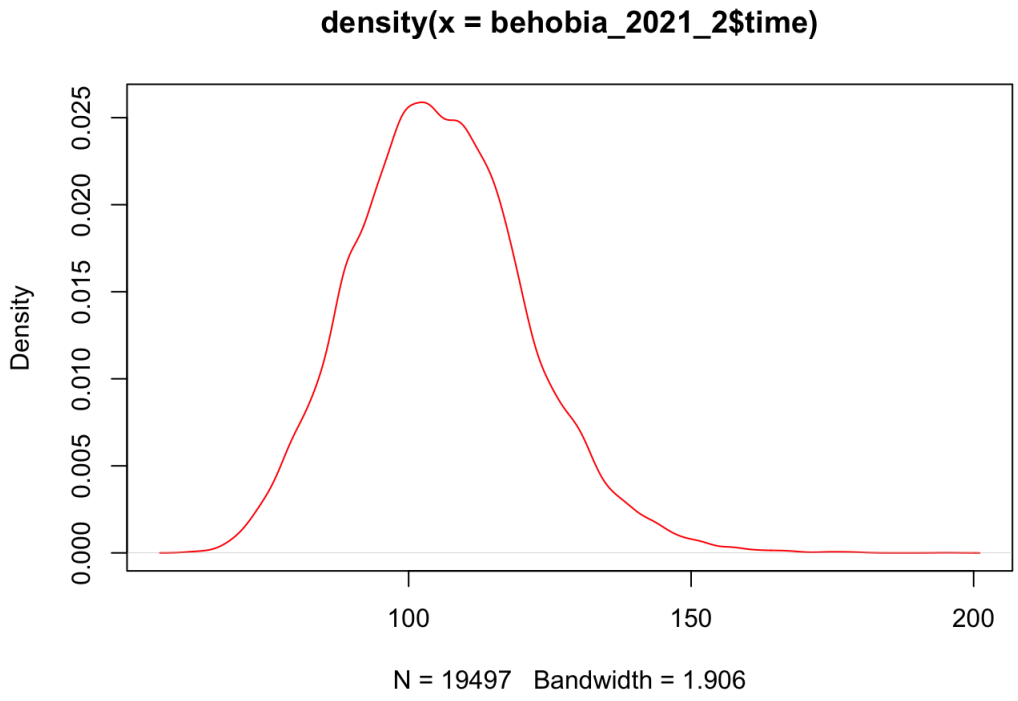
4 Densidad

Muy parecidas a los histogramas. Imaginar un histograma en el los `break` es igual a infinito. Esto correspondería a dibujar la función de densidad de mis datos.

R base

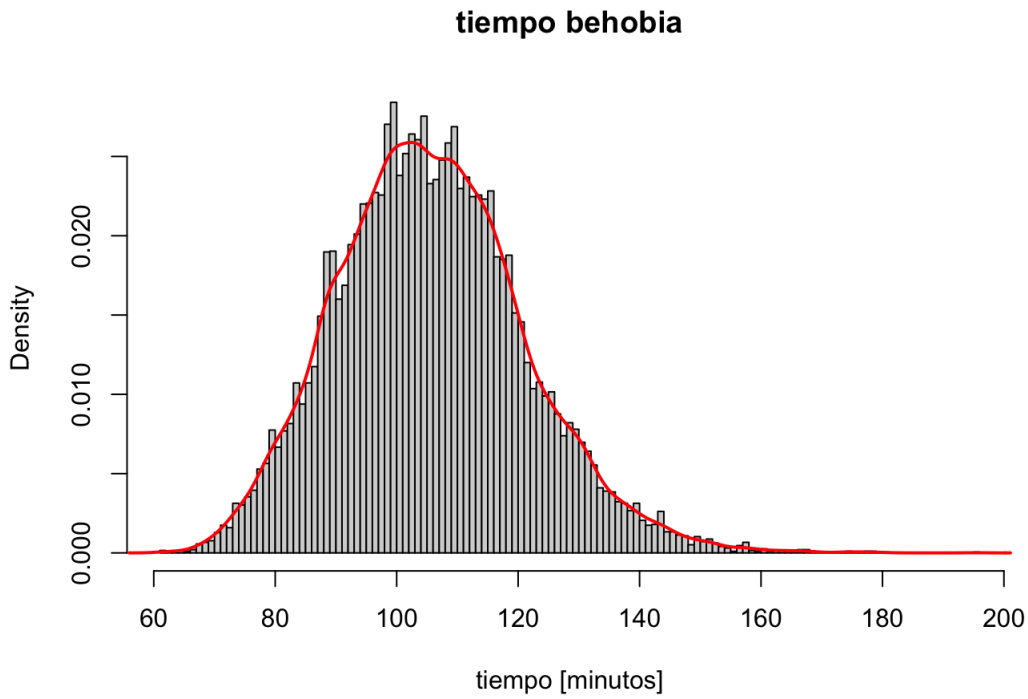
Para dibujar la función de densidad de un vector se puede utilizar la función `plot(density(vector))`.

```
plot(density(behobia_2021_2$time),col="red")
```



Para comparar el histograma con la función de densidad, vamos a dibujar en la misma gráfica el histograma que hemos dibujado antes y el "density plot". Para ello vamos a dibujar primero el histograma y luego vamos a añadir la función de densidad utilizando la función `lines`. Si en lugar de utilizar la función `plot` utilizamos la función `lines`, en lugar de empezar una gráfica nueva, dibujamos sobre la gráfica que ya existe.

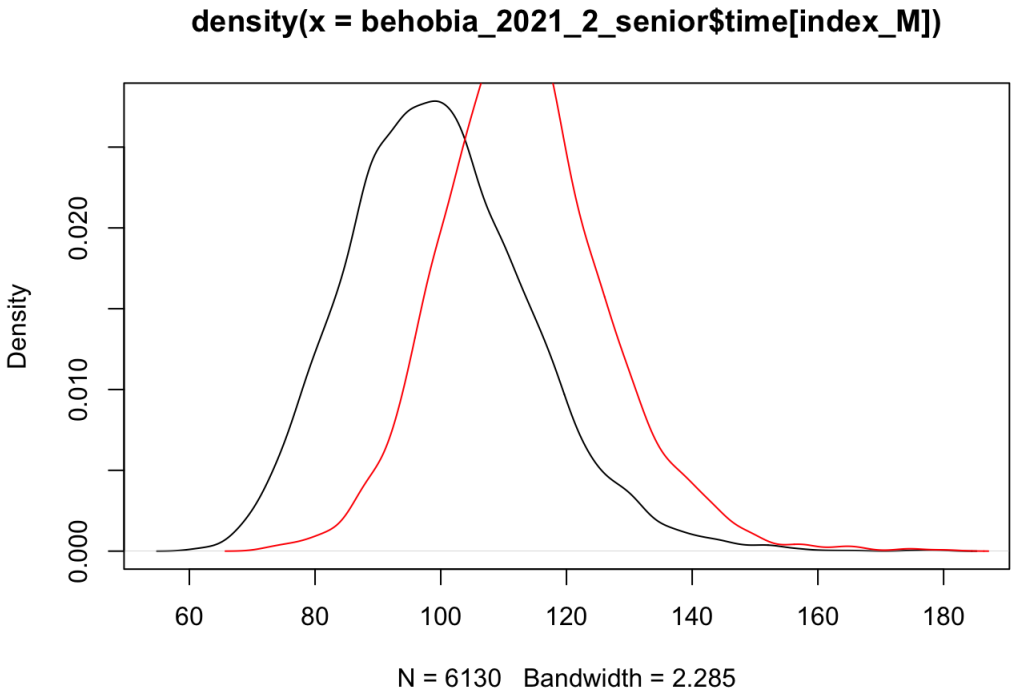
```
hist(x=behobia_2021_2$time,breaks = 100,main="tiempo behobia",xlab="tiempo [minutos]",fre
q = FALSE)
lines(density(behobia_2021_2$time),col="red",lwd=2)
```



```
# lwd es un parámetro para el grosor de la línea. Hemos puseto lwd=2
# para que la línea sea más gruesa y se vea mejor.
```

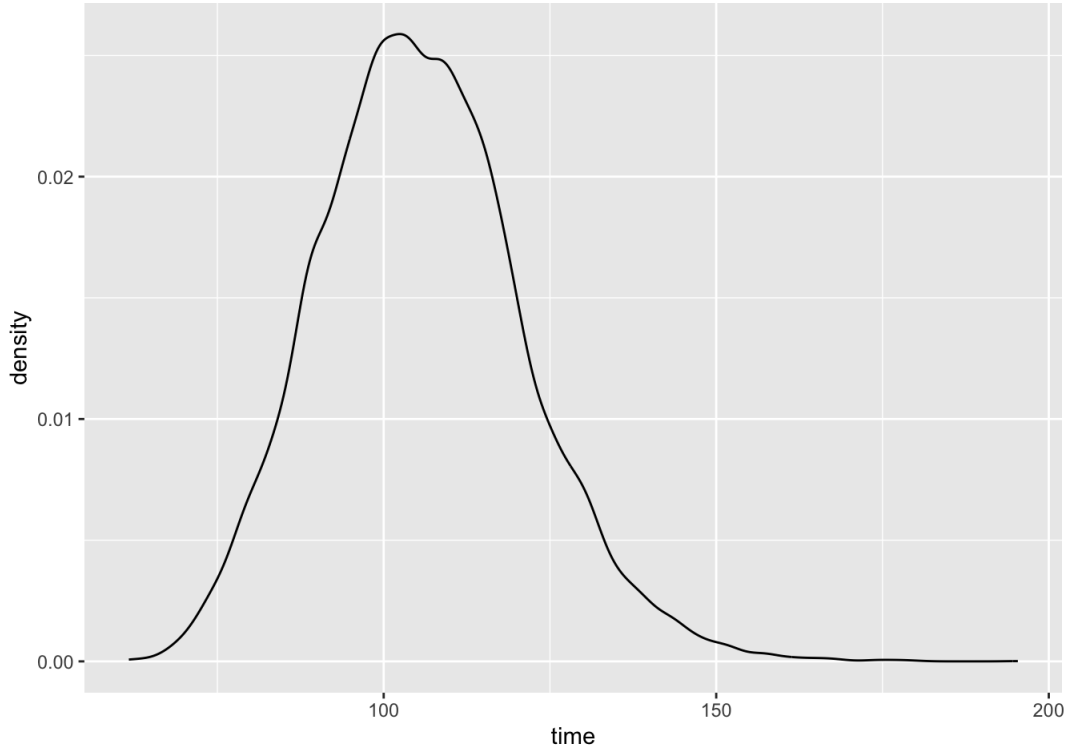
Al igual que antes, se puede dibujar 2 density plots para comparar 2 categorías:

```
index_M = behobia_2021_2_senior$categoria=="SENIOR M"
index_F = behobia_2021_2_senior$categoria=="SENIOR F"
plot(density(behobia_2021_2_senior$time[index_M]))
lines(density(behobia_2021_2_senior$time[index_F]),col="red")
```

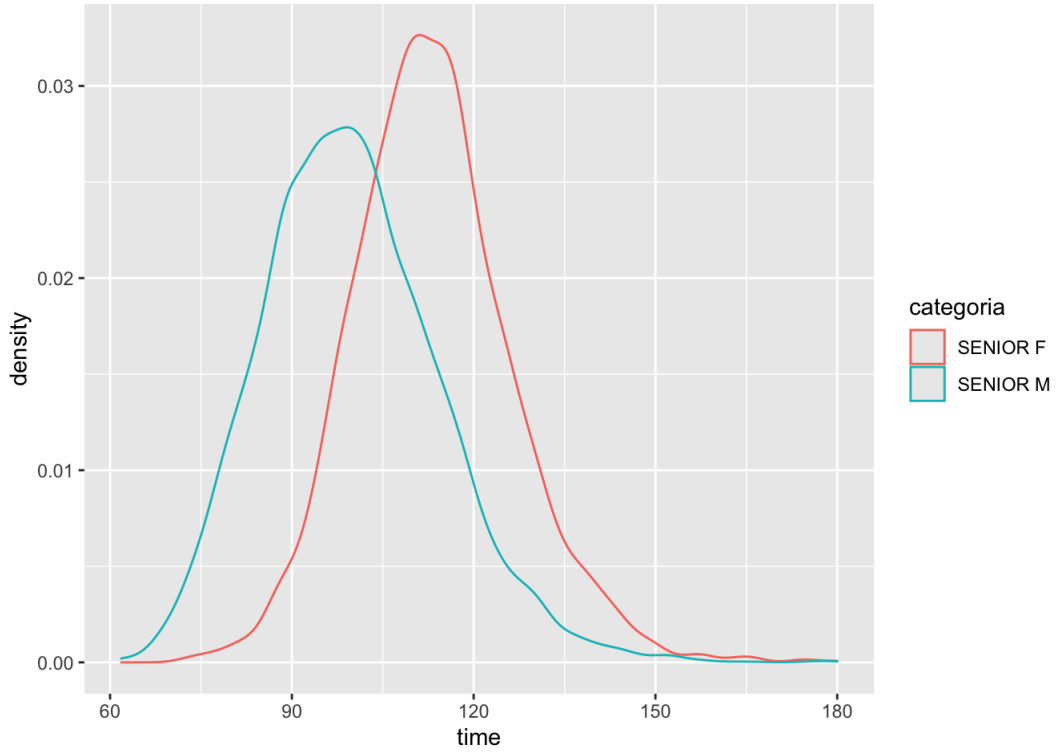


ggplot2

```
ggplot(data=behobia_2021_2,aes(x=time))+
  geom_density()
```




```
ggplot(data=behobia_2021_2_senior,aes(x=time,col=categoria))+
  geom_density()
```



5 Diagrama de barras

El diagrama de barras se utiliza para **representar variables discretas**.

Si en el eje Y de esta gráfica se representa la frecuencia, el gráfico se corresponde con la función de masa de la variable. Cuidado con confundir esta gráfica con un histograma. En caso de que la variable categórica tenga muchas categorías se parece mucho a un histograma.

También se puede utilizar, en algunos casos, para ver la **relación entre dos variables discretas**.

R base

La función que vamos a utilizar es la función `barplot`. Esta función tiene el siguiente parámetro de entrada:

- `height`: el valor en el eje y para cada valor discreto.

Vamos a ver el caso del resultado de sumar el valor de dos dados lanzamos de forma aleatoria e independiente:

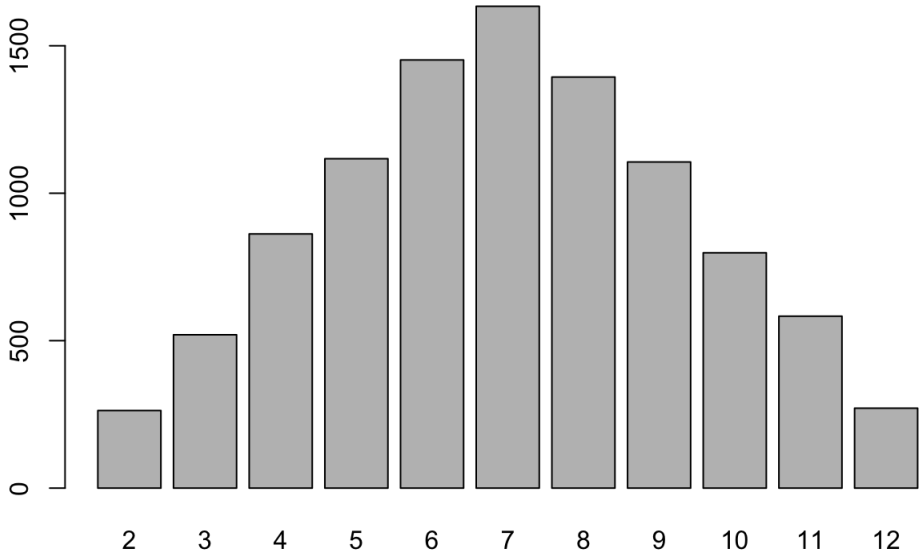
```
# con sample obtengo valores aleatorios de vector de
# entrada. En este caso quiero 10000 valores aleatorios
# de un vector que va del 1 al 6 (es como lanzar un dado
# 10000 veces)
dado1 <- sample(1:6,size=10000,replace = TRUE)
dado2 <- sample(1:6,size=10000,replace = TRUE)
resultados <- dado1 + dado2
```

Para obtener para cada posible resultado su correspondiente cantidad de repeticiones, podemos utilizar la función `table`:

```
cantidad <- table(resultados)
```

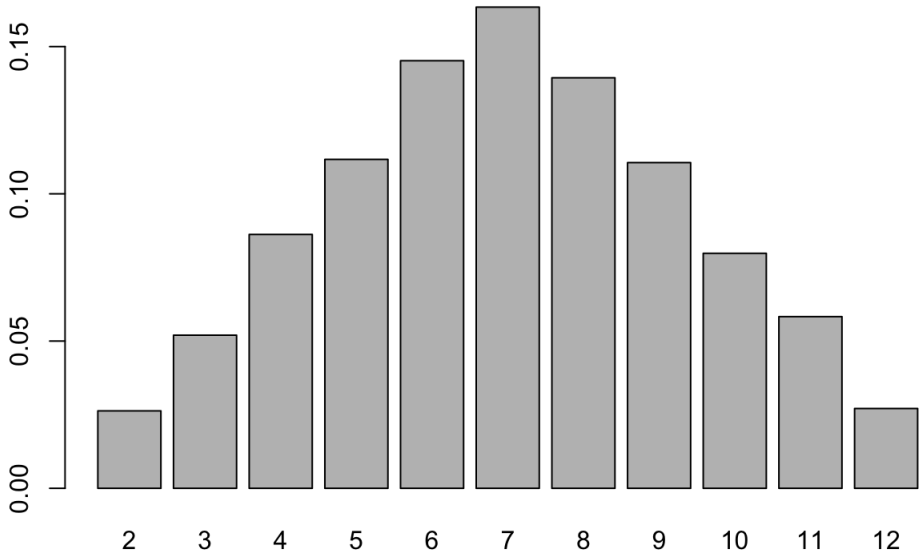
La variable `cantidad` será nuestro input de entrada en la función `barplot`:

```
barplot(height = cantidad)
```



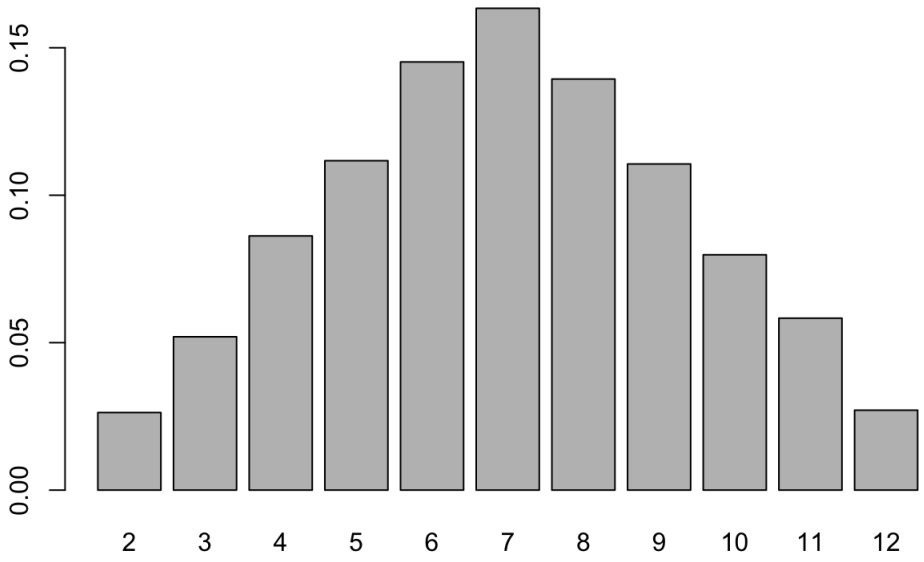
Para que esto sea equivalente a la función de masa, tenemos que dibujar la frecuencia de cada resultado. Para ello podemos dividir los valores de `frecuanci_res` entre el número de casos totales:

```
barplot(height = cantidad/10000)
```



En lugar de dividir entre 10000, podeos utilizar la función `prop.table`.

```
barplot(height = prop.table(cantidad))
```



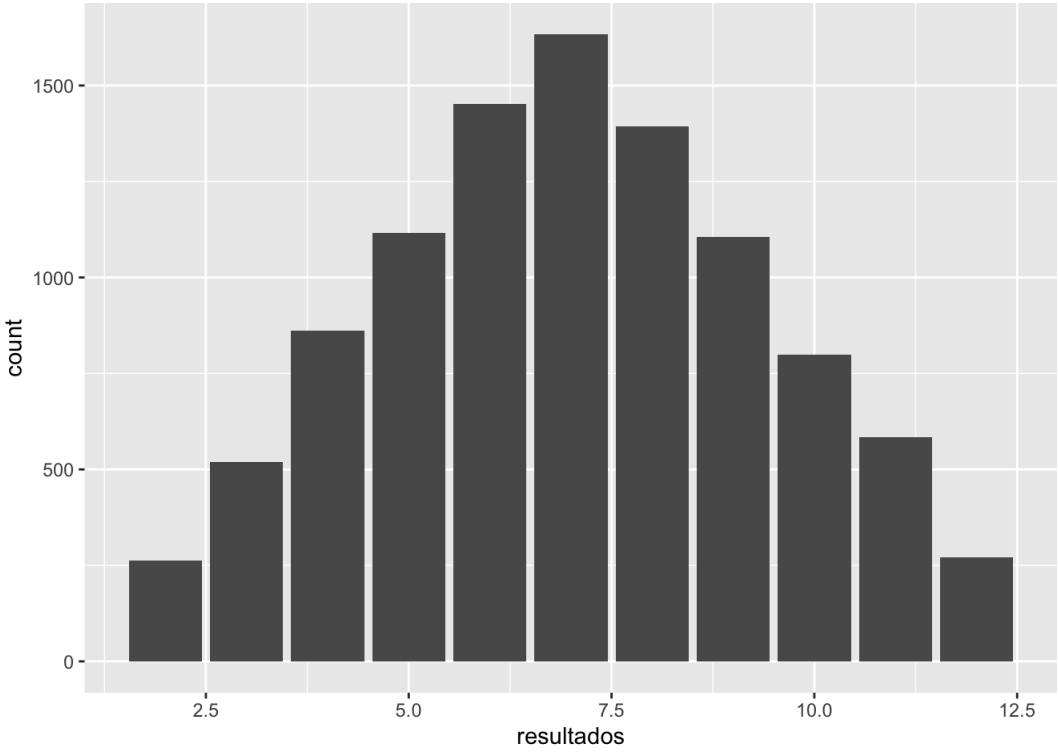
ggplot 2

Para poder trabajar con ggplot tenemos que tener un data.frame:

```
df = data.frame(cantidad=resultados)
```

Una vez tenemos el data.frame, podemos crear la gráfica:

```
ggplot(df,aes(x=resultados))+
  geom_bar()
```



5.0.1 relación 2 variables discretas

Volvamos al caso de la behobia. Vamos a trabajar con 2 variables categórias:

- si el corredor es SENIOR o JUNIOR.
- si el corredor ha tardado más o menos de 105 minutos.

Objetivo: observar si hay relación entre ser SENIOR o JUNIOR y tardar más o menos de 105 minutos.

Hay que darse cuenta de que esta información no está de forma explicita en el data.frame. Para obtener estos valores y poder graficar vamos a tener que modificar el data.frame o crearnos uno nuevo. Para esto es importante dominar las prácticas 1 y 2.

Esto se puede hacer de muchas maneras. Vamos a ver una de ellas. Si le preguntáis a chatGPT posiblemente el código que proponga será muy distinto.

R base

Vamos a ir paso a paso:

- vamos a crear un vector que nos indique que filas del data.frame corresponde a corredores SENIOR y corredores JUNIOR. Esto lo vamos a hacer con la función `which`. En la columna categoría tengo esta información, y por lo tanto quiero los corredores que sean "SENIOR F" , "SENIOR M", "JUNIOR F" y "JUNIOR M"

```
indice_senior = which(behobia_2021_2$categoria=="SENIOR F" | behobia_2021_2$categoria=="S
ENIOR M")
indice_junior = which(behobia_2021_2$categoria=="JUNIOR F" | behobia_2021_2$categoria=="J
UNIOR M")
```

- vamos a crear una nueva columna llamada "new_category" donde vamos a poner "SENIOR" a los corredores que sabemos que son SENIOR y JUNIOR a los que sabemos que son "JUNIOR"

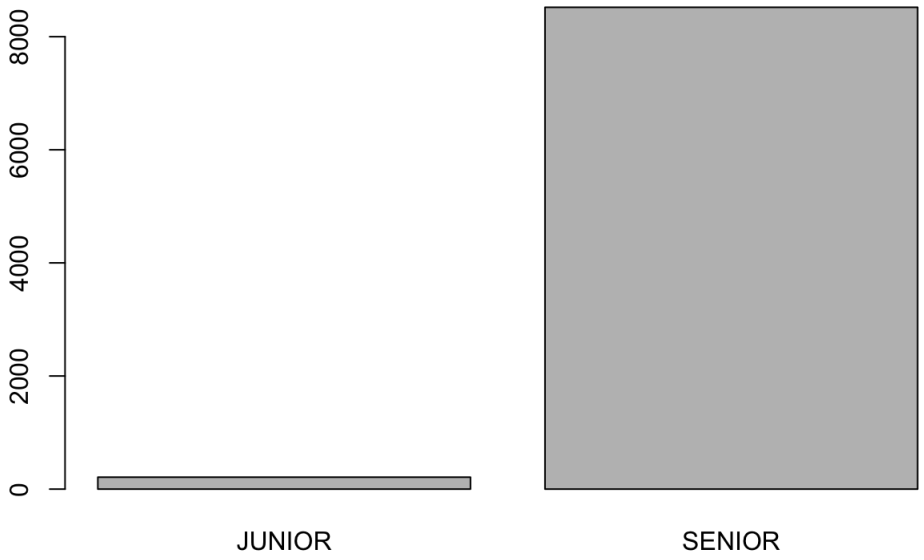
```
#creo la nueva columna vacía
behobia_2021_2$new_category = ""
behobia_2021_2$new_category[indice_junior] = "JUNIOR"
behobia_2021_2$new_category[indice_senior] = "SENIOR"
```

- Creo un nuevo data.frame con las filas de los corredores JUNIOR y SENIOR:

```
behobia_2021_junior_senior = behobia_2021_2[c(indice_junior,indice_senior),]
```

Por lo tanto, tengo en mi nuevo data.frame solo los corredores junior y senior:

```
barplot(table(behobia_2021_junior_senior$new_category))
```



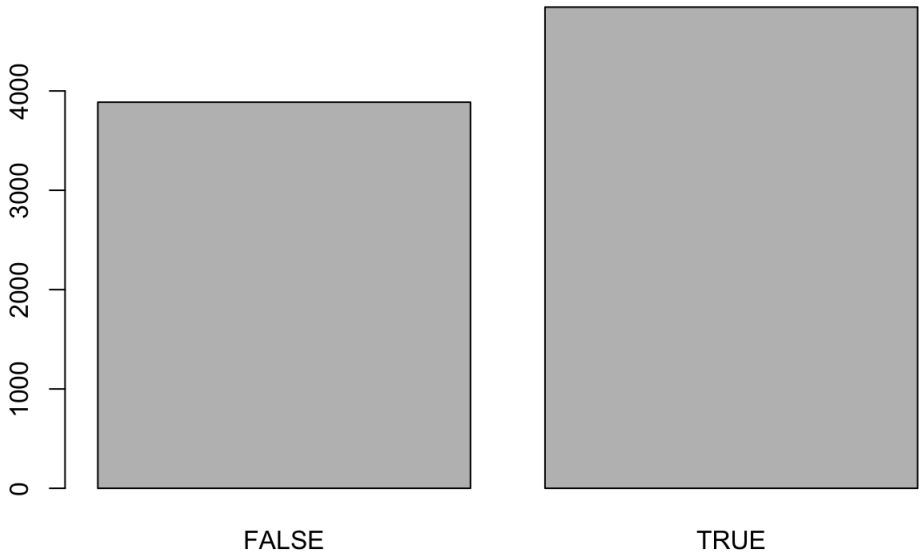
Como se observa en la gráfica tenemos muchos más casos de SENIOR que de JUNIOR.

- Nos falta crear la información de que corredores han tardado menos de 105 minutos. Para esto, vamos a crear una nueva columna “time_menor_105” que valdrá TRUE si el corredor ha tardado menos y FALSE en el caso contrario.

```
behobia_2021_junior_senior$time_menor_105 =  behobia_2021_junior_senior$time<105
```

Ahora podemos dibujar cuantos casos tenemos que han tardado menos de 105:

```
barplot(table(behobia_2021_junior_senior$time_menor_105))
```



Bastante parejo, tenemos casi el mismo número de corredores que han tardado menos de 105 minutos que los que han tardado más.

Ahora vamos a graficar para observar si hay relación entre ser SENIOR o JUNIOR y tardar más o menos de 105 minutos.

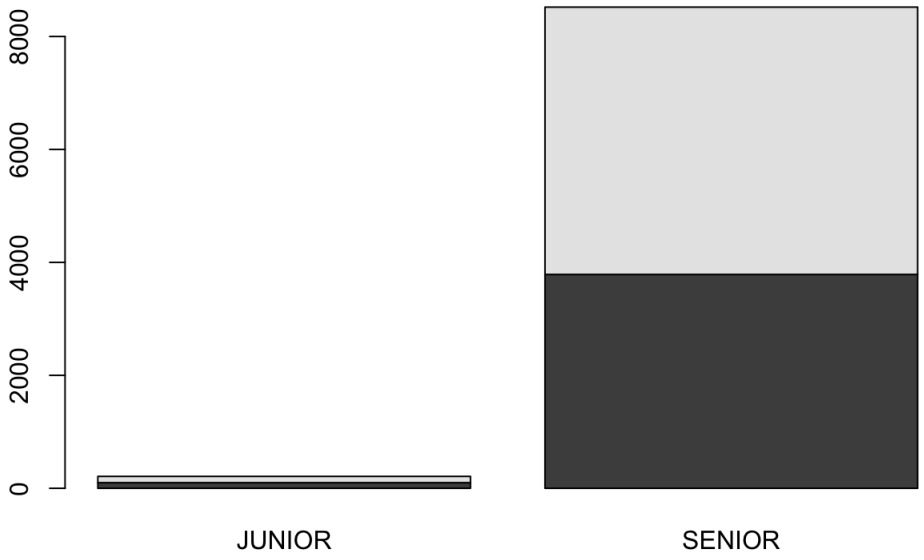
- primero aplicamos la función table:

```
tabla_contingencia = table(behobia_2021_junior_senior$time_menor_105,
                           behobia_2021_junior_senior$new_category)
tabla_contingencia
```

```
##
##          JUNIOR SENIOR
##   FALSE     100   3787
##   TRUE      111   4733
```

Tenemos que el junior 100 tardaron más de 105 minutos y 111 menos de 105 minutos. En SENIOR tenemos 3787 corredores que tardaron más y 4733 que tardaron menos.

```
barplot(tabla_contingencia)
```



Este diagrama se conoce como "diagrama de barras apilada" ¿Con esta gráfica se puede observar si hay relación? No facilmente. Para observar si hay relación o no, es mejor dibujar dibrama de barras aplidada proporcinal. Esto lo podemos dibujar utilizando la función `prop.table`.

La función `prop.table` tiene un parámetro `margin` que indica como queremos que calcule las proporciones. Si no se indica nada calcula el porcentaje total para los 4 casos:

```
prop.table(tabla_contingencia)
```

```
##
##             JUNIOR     SENIOR
##   FALSE 0.01145344 0.43374184
##   TRUE  0.01271332 0.54209140
```

En este caso, la suma de los 4 valores tiene que ser igual a 1.

Si ponemos `margin = 1` calcula para cada fila

```
prop.table(tabla_contingencia,margin = 1)
```

```
##
##             JUNIOR     SENIOR
##   FALSE 0.02572678 0.97427322
##   TRUE  0.02291495 0.97708505
```

En este caso, son las filas las que tienen que sumar 1. Las probabilidades que aparecen son probabilidades condicionadas.

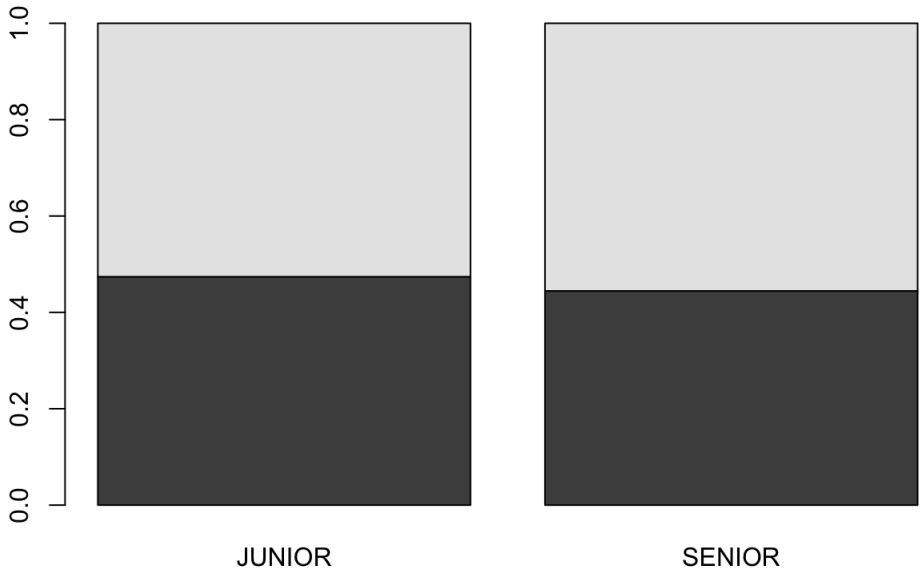
Y por último, con `margin = 2` calcula para cada columna:

```
prop.table(tabla_contingencia,margin = 2)
```

```
##
##             JUNIOR    SENIOR
##   FALSE 0.4739336 0.4444836
##   TRUE  0.5260664 0.5555164
```

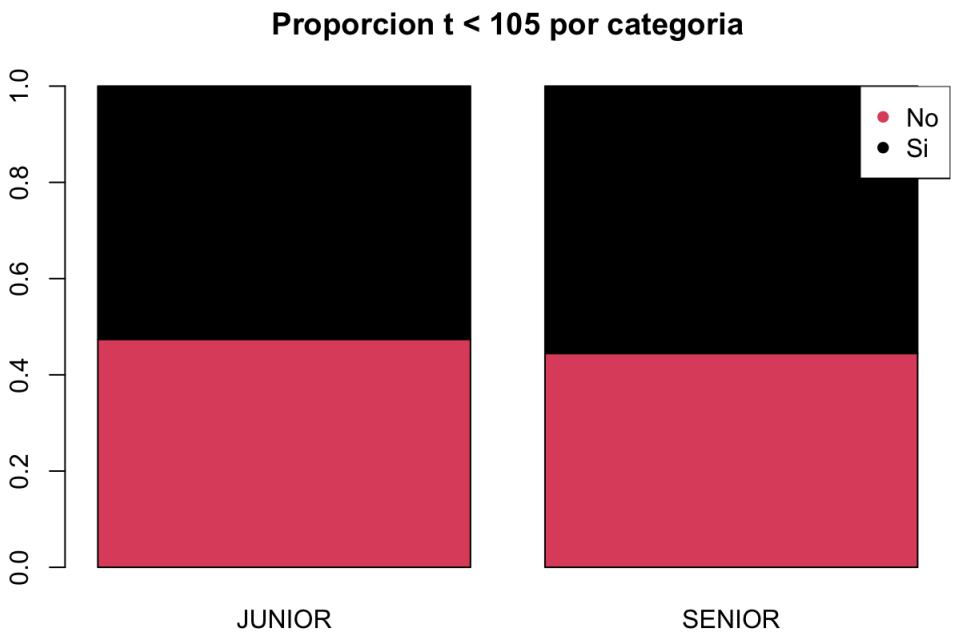
En este caso, son las columnas las que suman 1. Este último caso es el que nos interesa:

```
barplot(prop.table(tabla_contingencia,margin = 2))
```



Para terminar de completar la gráfica es importante añadir una leyenda que explique es signifca cada color. También habría que poner un título. ESTO EN EL EXAMEN ES IMPORTANTE.

```
barplot(prop.table(tabla_contingencia,margin = 2),
        col=c(2,1),main = "Proporcion t < 105 por categoria")
legend("topright",legend = c("No","Si"),
       col=c(2,1),pch=16)
```

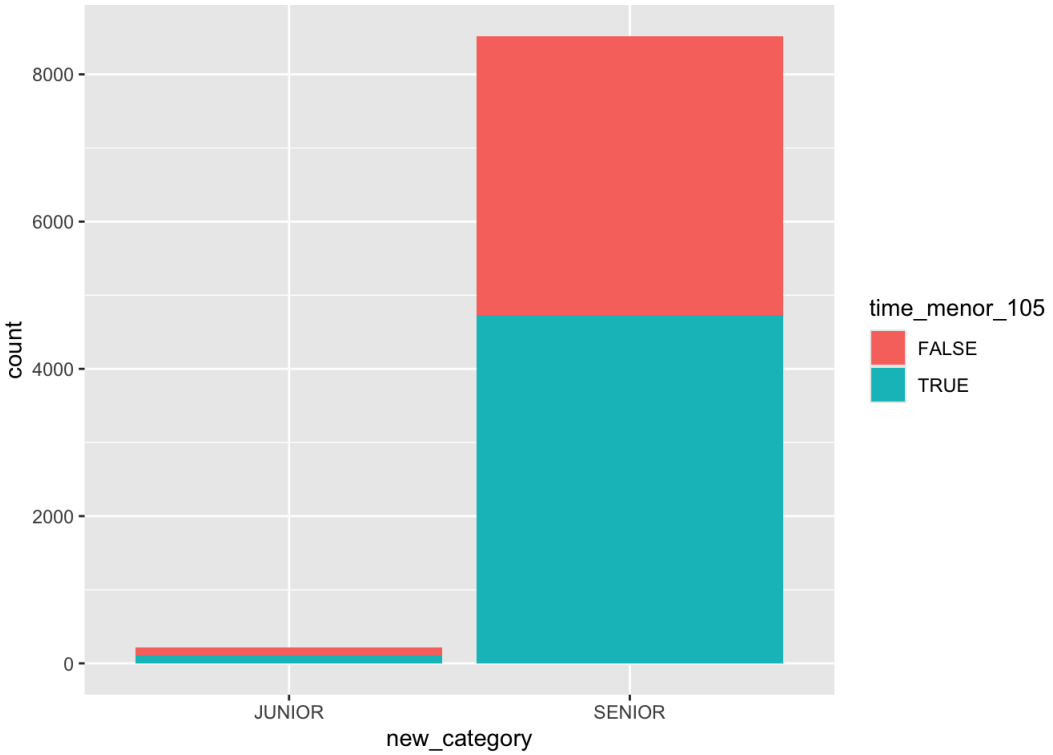


ggplot 2

Vamos a trabajar directamente con el data.frame `behobia_2021_junior_senior`

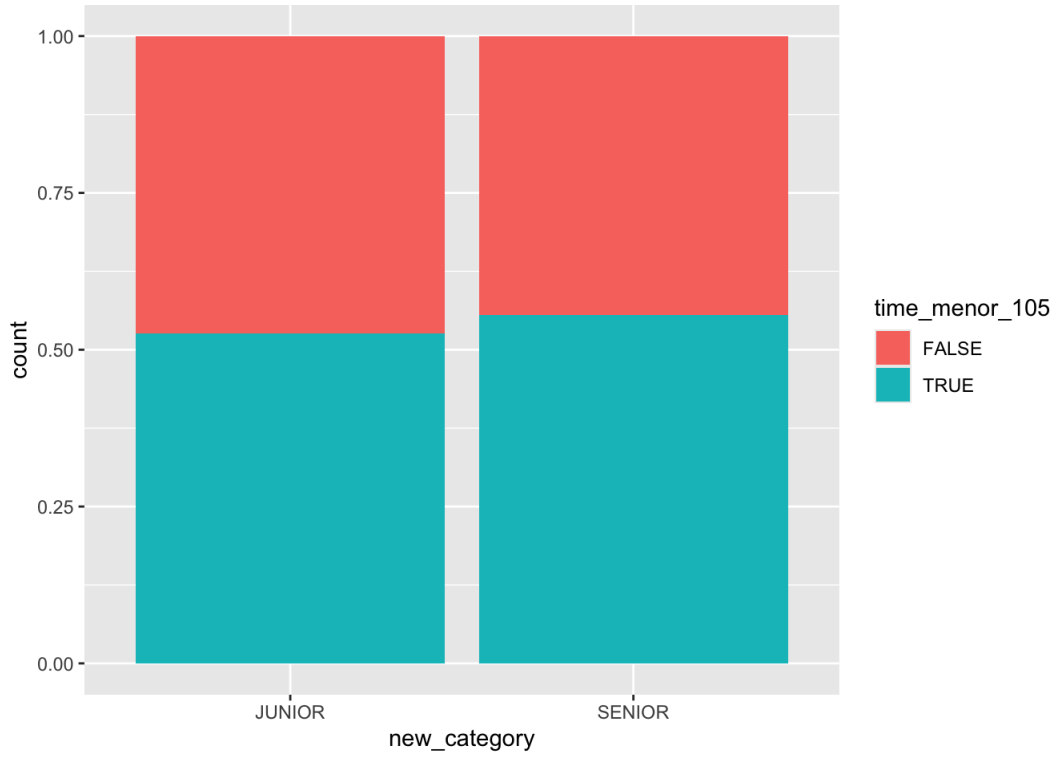
Para diabrama de barras apilado

```
ggplot(behobia_2021_junior_senior,aes(x=new_category,fill=time_menor_105))+
  geom_bar()
```



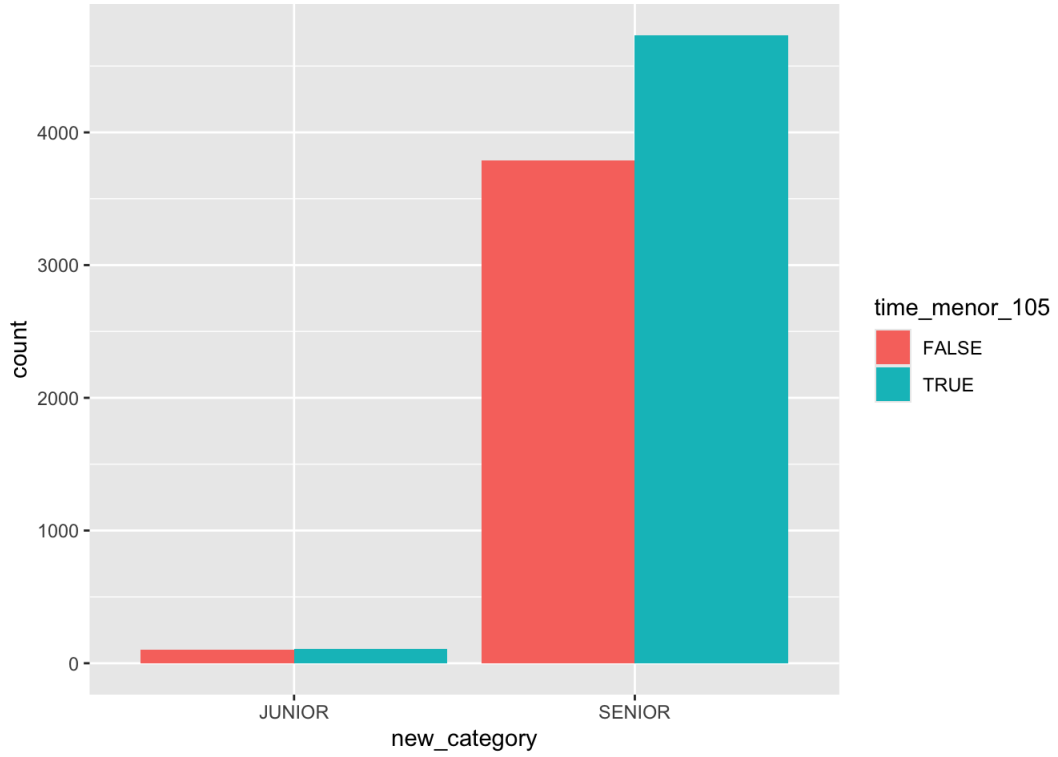
Para el diagrama de barras apilado proporcional:

```
ggplot(behobia_2021_junior_senior,aes(x=new_category,fill=time_menor_105))+
  geom_bar(position="fill")
```



Con `ggplot2` es más fácil que con R base crear otro tipo de gráficas:

```
ggplot(behobia_2021_junior_senior,aes(x=new_category,fill=time_menor_105))+
  geom_bar(position=position_dodge())
```



Aunque esta gráfica está bien, para observar relaciones es mejor la anterior.

6 mejorar ggplot

Hay muchísima información en internet sobre `ggplot2`.

En este link tenéis más información sobre gráficas en R, además de `ggplot2`

Como ya hemos visto, con ggplot hay 2 partes principales:

- `ggplot(data, aes(...))` donde indicamos cual es nuestro data.frame y las características principales de la gráfica: que hay que cada eje, en funciónd de que valores queremos que divida los datos y cómo queremos que los divida.

A modo de ejemplo. Queremos relacionar time total y tiempo en el km 5. Queremos que con el color nos distinga entre las categorias. También se podría con shape, etc.

```
ggplot(data=behobia_2021_2_senior,aes(x=time,y=time_km_5,col=categoria ))+
  geom_point()
```

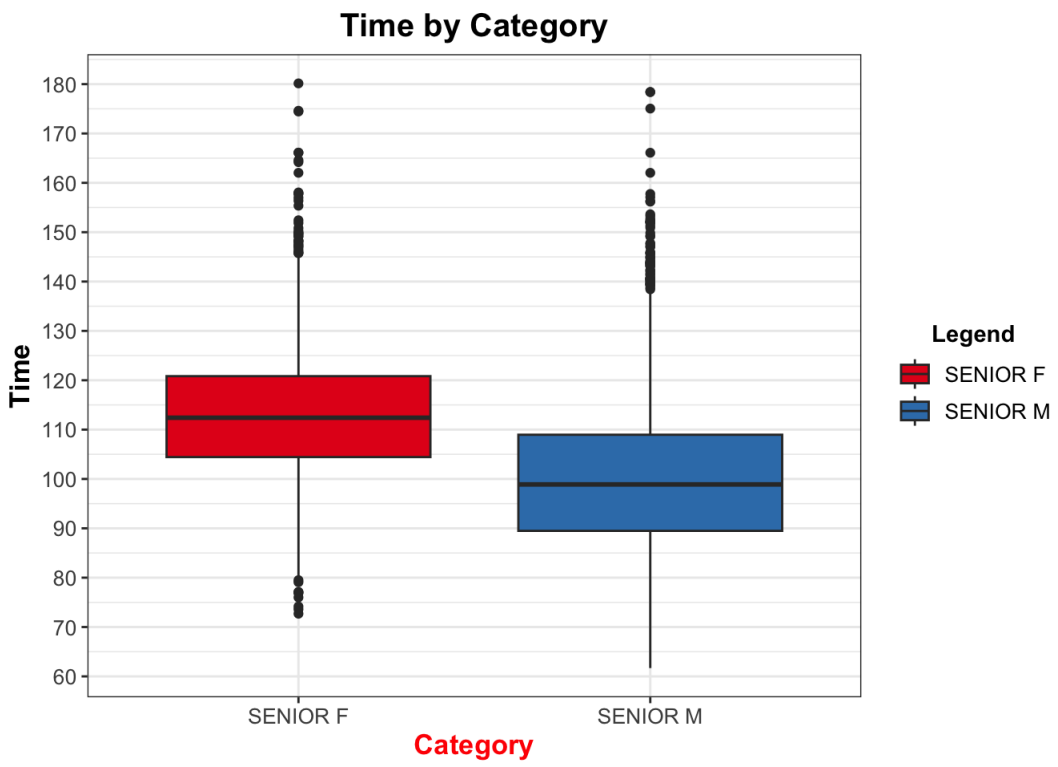
Y por otro lado el tipo de gráfica que queremos hacer.

- geom_point para scatter plot.
- geom_boxplot para boxplot.
- geom_histogram para histogramas.
- geom_density para densidad
- geom_bar para diagrama de barras.

Además, hay diferentes funciones que podemos utilizar para modificar la gráfica:

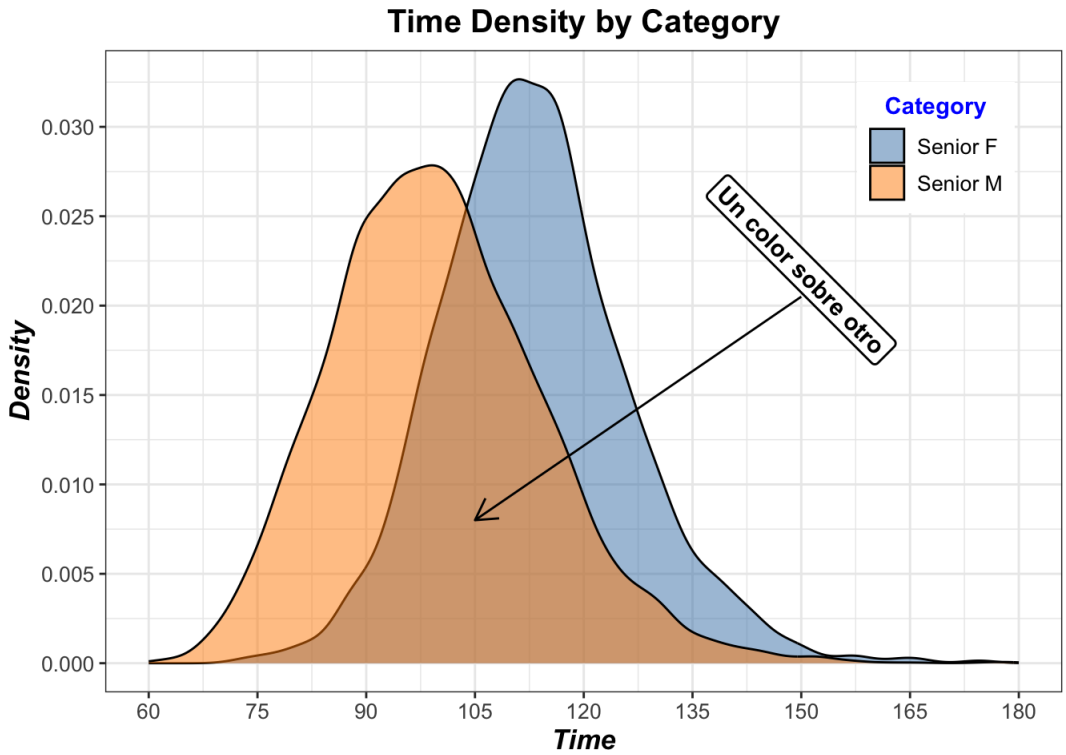
- Con la función `theme_bw` podemos modificar el estilo de la gráfica (existen más, como `theme_classic`)
- Con `theme` podemos modificar los ejes
- Con `ggtitle` podemos cambiar el título

```r
library(RColorBrewer) # para usar palettes

ggplot(data = behobia_2021_2_senior, aes(y = time, x = categoria, fill = categoria)) +
  geom_boxplot() +
  scale_fill_brewer(palette = "Set1") +  # puedes cambiar a "Pastel1", "Dark2", etc. (son
 palettes específicos de colores)
  scale_y_continuous(breaks = scales::pretty_breaks(n = 10)) +  # más intervalos en eje y
  ggtitle("Time by Category") +
  theme_bw() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 15),  # título centr
ado
        axis.title = element_text(face = "bold", size = 13),  # agrandar letras ejes
        axis.title.x = element_text(colour = "red"),
        axis.text = element_text(size = 10),  # tamaño texto
        legend.title = element_text(face = "bold", hjust = 0.5),  # título leyenda centra
do
        legend.text = element_text(size = 10)
        ) +
  labs(x = "Category", y = "Time", fill = "Legend")  # nombres de ejes y leyenda
```



```r
library(grid)  # para flechas con annotation_custom

ggplot(data = behobia_2021_2_senior,aes(x = time, fill = categoria)) +
  geom_density(alpha = 0.5) +
  scale_fill_manual(values = c("steelblue", "darkorange"),
                    labels = c("Senior F", "Senior M")) +  # Colores manuales y cambiamos
 nombre de labels
  scale_x_continuous(name = "Time", breaks = seq(60, 180, by = 15), limits = c(60, 180))
+ # Más intervalos en eje x (cada 15). OJO: Elimina un valor del grafico, porque no cumpl
e con el rango establecido
  scale_y_continuous(name = "Density", breaks = scales::pretty_breaks(n = 8)) +
  theme_bw() +
  ggtitle("Time Density by Category") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 15),
        axis.title = element_text(face = "bold.italic", size = 13),
        #axis.title.x = element_text(colour = "red"),
        axis.text = element_text(size = 10),
        legend.position = "inside",  # Oculta leyenda fuera del gráfico
        legend.position.inside = c(0.95, 0.95),  # La ubica dentro
        legend.justification = c("right", "top"),
        legend.title = element_text(face = "bold", colour = "blue", hjust = 0.5),
        legend.text = element_text(size = 10)
        ) +
  labs(x = "Time", y = "Density", fill = "Category") +
  annotate("label", x = 150, y = 0.022, label = "Un color sobre otro",
         angle = -45, fontface = "bold", size = 4,
         fill = "white", color = "black", label.size = 0.5) +  # Agrega texto dentro del
gráfico en lugar indicado
  annotate("segment", x = 150, xend = 105, y = 0.0205, yend = 0.008,
           arrow = arrow(length = unit(0.4, "cm")), color = "black") # Agrega flecha dent
ro del gráfico en lugar indicado
```

```
## Warning: Removed 1 row containing non-finite outside the scale range
## (`stat_density()`).
```



7 Ejercicios propuestos

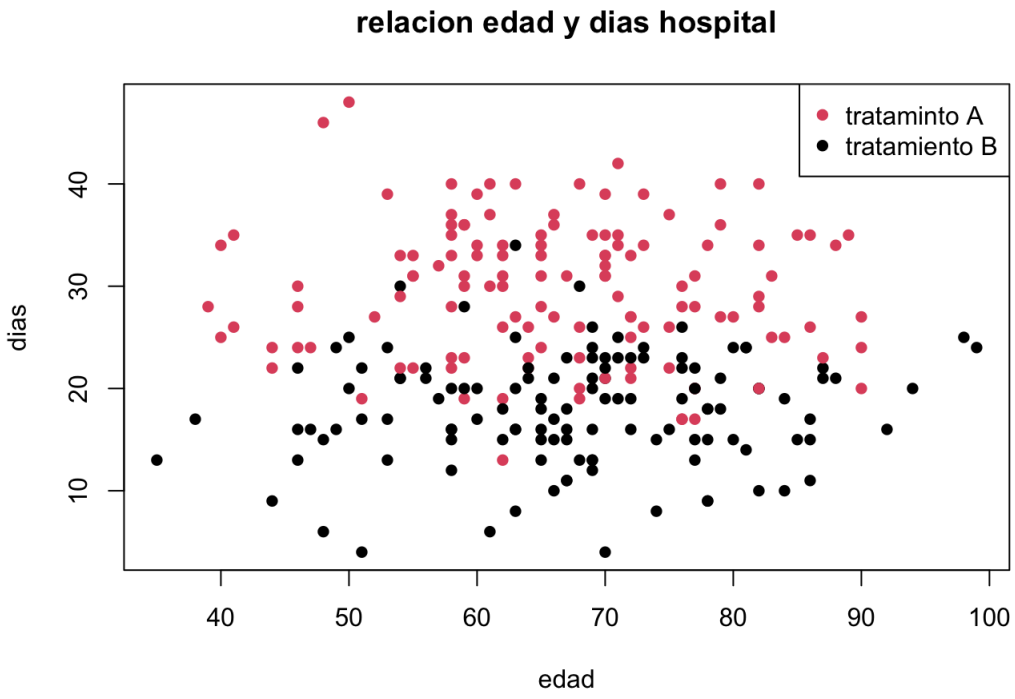
Para realizar estos ejercicios es necesario cargar el archivo `CORONAVIRUS.xlsx` que se encuentra en ADI. Para cargar este archivo pueden utilizar el siguiente código:

```
library(readxl)
data <- read_xlsx('./CORONAVIRUS.xlsx')
```

Estos ejercicios pueden tener más de una solución correcta. Para contestarlas realice las gráfica que crea convenientes.

1. ¿Hay relación entre la edad de los pacientes y el número de días que está hospitalizado? ¿Afecta a esta relación el tratamiento recibido?

```
color <- data$Tratamiento == "A"
color <- color+1

plot(data$Edad,
     data$Dias_hospital,col=color,pch=16,
     main="relacion edad y dias hospital",
     ylab="dias",xlab="edad")
legend("topright",legend = c("trataminto A","tratamiento B"),
       col=c(2,1),pch=16)
```




```
# la función legend sirve para mostrar la leyenda que nos indica que significan los colores
```

2. ¿Están relacionados el numero de plaquetas y la pleocitosis? Se ven diferencias en esa relaciónn en función del tipo de virus?
3. ¿Están relacionadas la pleocitosis y el número de leucocitos?
4. ¿Cómo se distribuyen los días de hospitalización?
5. ¿Cómo se distribuyen los días de hospitalicación en función del tratamiento?
6. ¿Cómo se distribuye el nivel de glucosa en función del sexo?
7. ¿Hay diferencias en los datos de hospitalización entre hombres y mujeres? ¿Y en función del tipo de virus?
8. ¿Hay diferencias en el nivel de glucosa entre los pacientes que tienen afasia y los que no?

9. ¿Cuál es el porcentaje de gente que ha recibido cada tratamiento?
10. Entro los pacientes que tienen nauseas, ¿Qué porcentaje de pacientes tienen cada tipo de virus?
11. ¿Cuántos pacientes tienen crisis?
12. En función del tipo de virus, ¿Cuántos pacientes tienen dolor de cabeza y cuántos no?